

CISS358: Algorithm Analysis Assignment 7

OBJECTIVES.

- To solve problems using the dynamic programming method.

Feel free to discuss among yourselves. Implementations must be in C++.

As always, if an execution is shown, it is to fix the input and output behavior of your program. The output need not be correct.

In your main.cpp, at the top wrote down the asymptotic runtime and space complexity:

```
// T(n) = O(?)
// SPACE(n) = O(?)

... your program ...
```

Create directory a07. Put the latex files in a07. For the program for a07/a07q02/ (main.cpp and other C++ source files or data files), put files in a07q02. Same for a07q03, etc.

SPRING2024: For now, the only mathematical writing is in q01.tex. For the other questions, you only need to implement C++ program. You are encouraged to try to write as many proofs for DP problems as you can – the proof of optimal substructure. After doing half a dozen you'll see that they are all more or less the same.

Q1. Prove that the rod cutting problem has the optimal substructure property. Here the statement of the problem:

ROD CUTTING PROBLEM 1. Let $P[1..k]$ (“prices”) be an array of positive values > 0 . ($P[i]$ is the price of a rod of length i . With this notation, you can extend the array P to index 0 by $P[0] = 0$ since the price of a rod of length 0 has to be 0.) Given an integer $n > 0$, find $\ell_0, \ell_1, \dots, \ell_{p-1}$ (the lengths of every cut; here, there are p pipes) such that

$$n = \ell_0 + \ell_1 + \dots + \ell_{p-1}, \quad 1 \leq \ell_i \leq k$$

and

$$\sum_{i=0}^{p-1} P[\ell_i]$$

is maximal.

SOLUTION.

Q2. Write a C++ program that solves the word segmentation problems.

Your program must include a trie (that's in CISS350). A file (filename `words.txt`) is provided for your dictionary. When loading each word in `words.txt` into your dictionary, convert all uppercase to lowercase and then ignore any word with non `a-z` characters. For instance ignore `1st` if it's in the file.

TEST 1

```
words.txt
toadsarenotfrogs
2
to ads are not frogs
toads are not frogs
```

The first input is the name of the file of words. The second input is the string to be segmented. This is followed by the number of segmentations and then all the segmentation; print a newline after a segmentation. If there are more than one segmentation, print all of them.

(I mentioned before that all books on algorithms only print one solution for a DP problem. The word segmentation problem is an example of a DP problem where you really should compute all solutions.)

Q3. Adobe is experimenting with a new image compression algorithm. A color picture is an $m \times n$ array $P[0..m-1][0..n-1]$ of pixels, where each pixel specifies a triple of red, green, and blue intensities. To compress this picture slightly, one pixel from each of the m rows is removed, so that the whole picture becomes one pixel narrower, i.e., the new array is going to be $m \times (n - 1)$. To avoid disturbing visual effects, pixels removed in two adjacent rows are in the same or adjacent columns. The collection of pixels removed is called a pixel-wide-cut.

To decide which pixels should be removed, for each pixel $P[i][j]$, an image processing algorithm has calculated a real-valued measure $u[i][j]$, indicating how bad the image file will be if $P[i][j]$ is removed. The lower the $u[i][j]$, the more similar the pixel is to its neighbors therefore the better it would be if $P[i][j]$ is removed. Removing $P[i][j]$ if $u[i][j]$ is high is bad. The sum total of all the $u[i][j]$ for the pixels $P[i][j]$ s in a pixel-wide-cut, is called the ugliness of that cut. The goal is to minimize the ugliness.

(a) How many pixel-wide-cuts are there for an $m \times n$ image file?

- (b) Give an algorithm to find pixel-wide-cuts with the minimal ugliness.
- (c) Implement your algorithm in C++.
- (d) What is the runtime and space complexity of your algorithm? Write this in the documentation of your main.cpp:

```
// T(n) = O(?)
// SPACE(n) = O(?)

... your program ...
```

Here's a sample output:

```
3 4
1.1 2.2 3.3 4.4
2.2 3.3 4.4 2.2
3.3 4.4 2.2 2.2
6.6
((0,0), (1,0), (2,0))
((0,0), (1,1), (2,2))
```

The first input is the number of rows, the second input is the number columns, this is followed by the values of d . The first output, 6.6, is the minimal ugliness. This is followed by the pixel-wide-cuts achieving the minimal ugliness. In the above, there are two possible cuts: the first is to remove pixels (0,0), (1,0), (2,0) and the second is to remove pixels (0,0), (1,1), (2, 0).

Q4. In order to transform one source string of text $x[1..m]$ to a target string $y[1..n]$, we can perform various transformation operations. Our goal is, given x and y , to produce a series of transformations that change x to y . We use an array z – assumed to be large enough to hold all the characters it will need – to hold the intermediate results. Initially, z is empty, and at termination, we should have $z[j] = y[j]$ for $j = 1, 2, \dots, n$. We maintain current indices i into x and j into z , and the operations are allowed to alter z and these indices. Initially, $i = j = 1$. We are required to examine every character in x during the transformation, which means that at the end of the sequence of transformation operations, we must have $i = m + 1$.

We may choose from among six transformation operations:

- Copy a character from x to z by setting $z[j] = x[i]$ and then incrementing both i and j . This operation examines $x[i]$.
- Replace a character from x by another character c , by setting $z[j] = c$, and then

incrementing both i and j . This operation examines $x[i]$.

- Delete a character from x by incrementing i but leaving j alone. This operation examines $x[i]$.
- Insert the character c into z by setting $z[j] = c$ and then incrementing j , but leaving i alone. This operation examines no characters of x .
- Twiddle (i.e., exchange) the next two characters by copying them from x to z but in the opposite order; we do so by setting $z[j] = x[i + 1]$ and $z[j + 1] = x[i]$ and then setting $i = i + 2$ and $j = j + 2$. This operation examines $x[i]$ and $x[i + 1]$.
- Kill the remainder of x by setting $i = m + 1$. This operation examines all characters in x that have not yet been examined. This operation, if performed, must be the final operation.

As an example, one way to transform the source string algorithm to the target string altruistic is to use the following sequence of operations, where the underlined characters are $x[i]$ and $z[j]$ after the operation:

Operation	x	z
initial strings	<u>a</u> lgorithm	-
copy	<u>a</u> lgorithm	a_
copy	<u>a</u> lgorithm	al_
replace by t	al <u>g</u> orithm	alt_
delete	algori <u>th</u> m	alt_
copy	algori <u>th</u> m	altr_
insert u	algori <u>th</u> m	altru_
insert i	algori <u>th</u> m	altrui_
insert s	algori <u>th</u> m	altruis_
twiddle	algori <u>th</u> m	altruisti_
insert c	algori <u>th</u> m	altruistic_
kill	algori <u>th</u> m_	altruistic_

Note that there are several other sequences of transformation operations that transform algorithm to altruistic.

Each of the transformation operations has an associated cost. The cost of an operation depends on the specific application, but we assume that each operation's cost is a constant that is known to us. We also assume that the individual costs of the copy and replace operations are less than the combined costs of the delete and insert operations; otherwise, the copy and replace operations would not be used. The cost of a given sequence of transformation operations is the sum of the costs of the individual operations in the sequence. For the

sequence above, the cost of transforming `algorithm` to `altruistic` is

$$3 \cdot \text{cost}(\text{copy}) + \text{cost}(\text{replace}) + \text{cost}(\text{delete}) + 4 \cdot \text{cost}(\text{insert}) \\ + \text{cost}(\text{twiddle}) + \text{cost}(\text{kill})$$

- (a) Given two sequences $x[1..m]$ and $y[1..n]$ and set of transformation-operation costs, the **edit distance** from x to y is the cost of the least expensive operation sequence that transforms x to y . Describe a dynamic-programming algorithm that finds the edit distance from $x[1..m]$ to $y[1..n]$ and prints an optimal operation sequence. Analyze the running time and space requirements of your algorithm. Write a C++ program implementing your algorithm. Here is a sample execution:

```

algorithm
altruistic
1 2 3 1 2 5
19
initial strings    >algorithm
copy              a>lgorithm    a
copy              al>gorithm    al
replace by t      alg>orithm    alt
delete            algo>rithm    alt
copy              algor>ithm    altr
insert u          algor>ithm    altru
insert i          algor>ithm    altrui
insert s          algor>ithm    altruis
twiddle           algorith>hm    altruisti
insert c          algorith>hm    altruistic
kill              algorithm>    altruistic

```

The first two inputs are the strings x and z . The next 6 inputs (integers) are the cost of copy, replace, delete, insert, twiddle, kill. The first output is the cost. The rest describe the operations.

The edit-distance problem generalizes the problem of aligning two DNA sequences. There are several methods for measuring the similarity of two DNA sequences by aligning them. One such method to align two sequences x and y consists of inserting spaces at arbitrary locations in the two sequences (including at either end) so that the resulting sequences x' and y' have the same length but do not have a space in the same position (i.e., for no position j are both $x'[j]$ and $y'[j]$ a space.) Then we assign a “score” to each position. Position j receives a score as follows:

- +1 if $x'[j] = y'[j]$ and neither is a space,
- -1 if $x'[j] \neq y'[j]$ and neither is a space,
- -2 if either $x'[j]$ or $y'[j]$ is a space.

The score for the alignment is the sum of the scores of the individual positions. For example, given the sequences $x = \text{GATCGGCAT}$ and $y = \text{CAATGTGAATC}$, one alignment is

```

G ATCG GCAT
CAAT GTGAATC
-*****+---*

```

A + under a position indicates a score of +1 for that position, a - indicates a score of -1, and a * indicates a score of -2, so that this alignment has a total score of $6 \cdot 1 - 2 \cdot 1 - 4 \cdot 2 = -4$.

- (b) Explain how to cast the problem of finding an optimal alignment as an edit distance problem using a subset of the transformation operations copy, replace, delete, insert, twiddle, and kill.

Q5. NSA is designing a new steganographic system to hide information. Here's how it works.

Suppose agent Bob wants to send a message **mad** to Alice and Eve the rogue agent is eavesdropping. Bob appends the characters of the message, except the last, to the message but in the reverse order to get **madam**. Note that this is a palindrome with odd length. He then inserts a huge number of characters into the string to further obfuscate the message. Here's an example:

hieveryhamburgerdoesnottastegoodwithmarmalade

The problem is Bob might accidentally create many subsequences in this obfuscated message. So Bob and Alice has agreed that if there is more than one possible subsequence, the longest one is the correct palindrome. NSA has asked you to write a program to find the longest odd-length palindromic subsequence of a given input string (hopefully there's only one). Of course it's possible that there are more than one longest odd-length palindrome subsequence. In that case, your program will be helpful in warning Bob that his obfuscation is ambiguous.

- (a) For a string of length n , how many subsequences are there altogether? (Using brute force, you can then check each subsequence to see if the length is odd and if it is a palindrome. Of course you know by now that brute force is a bad idea.)

- (b) Define a suitable measure for optimization (a length function) in order to solve this problem. For instance if the input is $x[0..n-1]$, maybe you want the function $L(i)$ to be the length of the longest palindromic subsequence of $x[0..i]$, or maybe you want the length of the longest palindromic subsequence of $x[i..n-1]$ or maybe you want the function is $L(i, j)$ to be the length of the longest palindromic subsequence of $x[i..j]$. Or maybe you want something that is completely different (and more crazy than what I can think of.)

- (c) Find a recursion on your function in (b). Implement a top-down DP solution.

- (d) Find a bottom-up DP solution to our problem and implement it in C++. Here's a sample run:

```
whatacharacter  
("carac", (5, 7, 8, 9, 10))
```

There is only one input. (Bob wanted to say **car**, so the palindrome is **carac**.) The output is the longest odd-length palindromic subsequence followed by the indices of this subsequence. If there are more than one longest palindromic subsequences, print them on separate lines. For instance in the above, there's another output:

```
whatacharacter  
("carac", (5, 7, 8, 9, 10))  
("hatah", (1, 2, 3, 4, 6))
```

(Note that for this part, you should print the longest. **tcaract** is actually longer. The above test case is only to fix the input-output format. Outputs shown above need not be correct.)

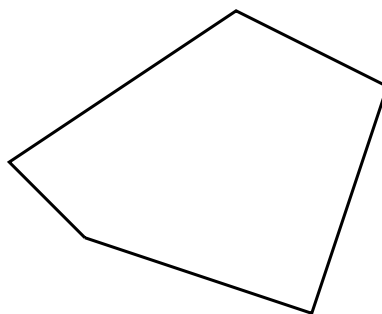
(e) What is the runtime and space complexity of your algorithm? Write this in the documentation of your main.cpp:

```
// T(n) = O(?)  
// SPACE(n) = O(?)  
  
... your program ...
```

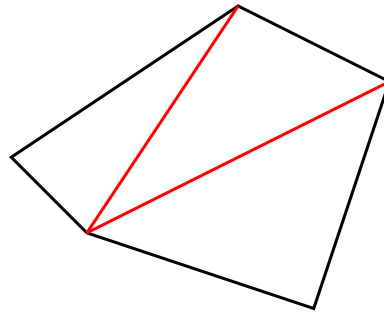
(f) Finally using the above program write another program that prints all odd-length palindromic subsequence with length of at least 3. Organized the output in ascending order by length. For string with the same length, organize them by dictionary order.

NOTE. Assume characters in string are in lowercase.

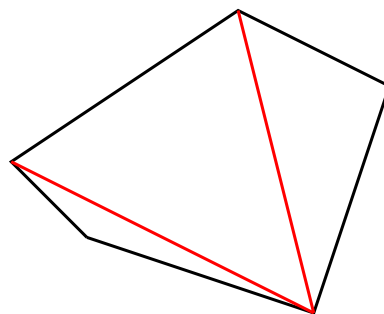
Q6. Polygonia is a high-end avant-garde architecture company. Some of their designs are based on convex polygons. For instance here's a convex polygon



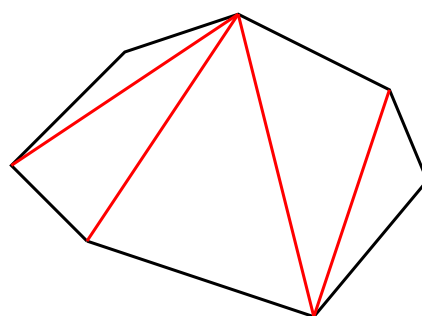
“Convex” means if you join two points of the polygon with a line segment, the line segment is inside the polygon. For course a fancy mansion has to be divided into rooms: bedrooms, banquet rooms, study rooms, libraries, swimming pool rooms, cinema rooms, etc. Polygonia is starting a new line of designs where the rooms are created by building interior walls (walls inside the building) joining vertices of a polygonal building. Here’s an example where there are two interior walls for the above building:



and here’s another:



Here’s another wall design for a more complicated building:



In other words a design of interior walls is a collection of non-intersecting line segments in the polygon building such that each line segment joins two vertices of the polygon and the resulting rooms must be triangles. The cost of interior walls is the sum of the lengths of the line segments in the architectural design (think Pythagorus theorem). The goal is to find the minimal cost for the interior walls when given the vertices of the building.

- (a) For a polygonal build with n vertices, how many different collections of interior wall design are there? (SOLUTION PROVIDED.)
- (b) Find a suitable recursion that helps solve this problem. (You have to find a function to optimize. Make sure you state the function clearly.)
- (c) Find and implement in C++ a bottom-up DP solution to the problem. Here's a sample execution

```
5
0.0 0.0 3.1 -1.0 4.2 2.0 2.1 3.1 -1.0 1.1
12.7
{{0,2}, {0,3}}
```

The first input is the number of vertices of the polygon. This is followed by the (x, y) -coordinates of the points listed in counter-clockwise direction. In the above, the polygonal is made up of points $p_0 = (0.0, 0.0)$, $p_1 = (3.1, -1.0)$, $p_2 = (4.2, 2.0)$, $p_3 = (2.1, 3.1)$, and $p_4 = (-1.0, 1.1)$. The output is 12.7 (the minimal cost of among all possible wall designs). This is followed by the optimal wall design. For instance in the above the first line wall is $\{0, 2\}$, meaning the line segment p_0, p_2 and the second line segment is $\{0, 3\}$ meaning p_0, p_3 . If there are two possible optimal wall designs, you must list all of them, one complete wall design on each line.

- (d) What is the runtime and the space complexity of your algorithm? Write in the documentation of your main.cpp:

```
// T(n) = O(?)
// SPACE(n) = O(?)

... your program ...
```

Solution and hint on the page next ...

WARNING ... INCOMING SOLUTION AND HINT

(a) Let $N(n)$ be the number of ways to triangulate an n -gon. Then

$$N(n) = N(3)N(n-1) + N(4)N(n-2) + N(5)N(n-3) + \cdots + N(n-1)N(3)$$

where $N(3) = 1$. Let $N'(n) = N(n+2)$. Then the above becomes

$$N'(n-2) = N'(1)N'(n-3) + N'(2)N'(n-4) + N'(3)N'(n-5) + \cdots + N'(n-3)N'(1)$$

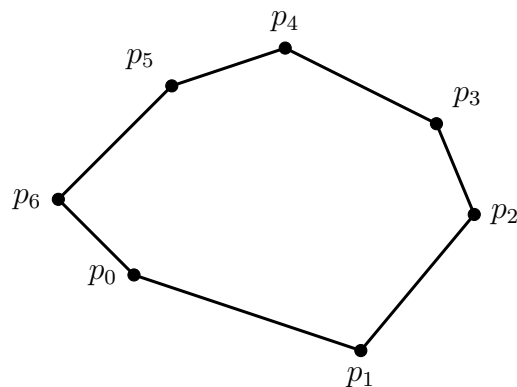
Therefore $N'(n-2)$ is C_{n-2} , the $(n-2)$ -th Catalan number. Hence

$$N(n) = N'(n-2) = C_{n-2} = \frac{1}{n-2} \binom{2(n-2-1)}{n-2} = \frac{1}{n-2} \binom{2(n-3)}{n-2}$$

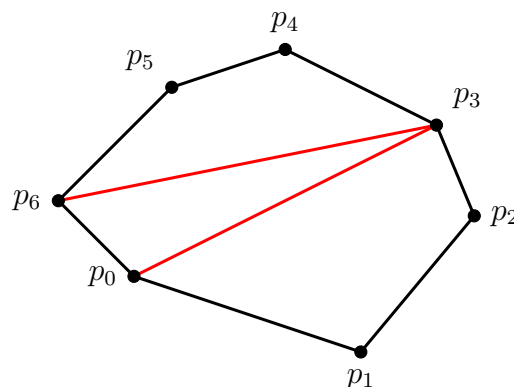
Asymptotically,

$$N(n) = C_{n-2} = O(4^{n-2}/\sqrt{n-2})$$

(c) Consider this polygon as an example:



Let $C(0, 6)$ to be the cost of the polygon. If I divide the polygon this way:



I can consider the cost $C(0, 3)$ and $C(3, 6)$ where $C(0, 3)$ is the cost for polygon $p_0p_1p_2p_3$ and $C(3, 6)$ is the cost for polygon $p_3p_4p_5p_6$.

Note that the cost $C(0, 6)$ is the cost of the diagonals in the $p_0...p_6$ polygon. Likewise the cost $C(0, 3)$ is the cost of the diagonals in the $p_0...p_3$ polygon. The cost of the line p_0p_3 is not included in $C(0, 3)$. Likewise the cost of the line p_3p_6 is not in the cost $C(3, 6)$. Therefore cost of triangulating $p_0...p_6$ where $p_0p_3p_6$ is chosen to be in the triangulation is

$$C(0, 3) + C(3, 6) + c(p_0, p_3) + c(p_3, p_6)$$

where $c(p_i, p_j)$ is the cost of the wall from p_i to p_j . Of course we have to go through all the possible cases:

$$C(0, 6) = \min\{C(0, k) + C(k, 6) + c(p_0, p_k) + c(p_k, p_6) \mid k = 1, 2, 3, \dots, 5\}$$

In general

$$C(i, j) = \min\{C(i, k) + C(k, j) + c(p_i, p_k) + c(p_k, p_j) \mid i + 1 \leq k \leq j - 1\}$$

It should be clear that

$$C(i, i + 1) = 0$$

(e) The runtime is $O(n^3)$ and the space is $O(n^2)$.